Flutter

Flutter'da Navigasyon ve Yönlendirme

Mobil Programlama I

Ekranlar Arası Geçiş ve Derin Bağlantı Yönetimi

HAZIRLAYANLAR Nilay Özçelik • Yüsra Paçal • Yağmur Öztekin

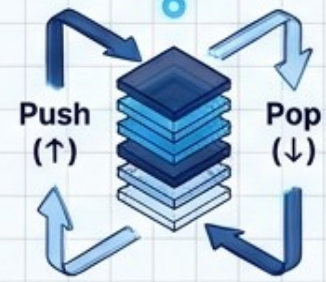
Uygulama İçi Hareketin Haritası

Bölüm 1: Temeller



Route ve Navigator Yığını

Temel navigasyon yapıları



Push & Pop İşlemleri

Yığına sayfa ekleme ve çıkarma

Bölüm 2: Arayüz ve Veri



go_router Mimarisi

Bildirimsel routing ve URL senkronizasyonu



TabBar ve Drawer

Kullanıcı arayüzü bileşenleri ve navigasyon



Ekranlar Arası Veri Aktarımı

Verilerin güvenli aktarımı ve state yönetimi

Bölüm 3: Dış Bağlantılar



Derin Bağlantı (Deep Linking)

Harici kaynaklardan uygulama içi yönlendirme



İşletim Sistemi Konfigürasyonları

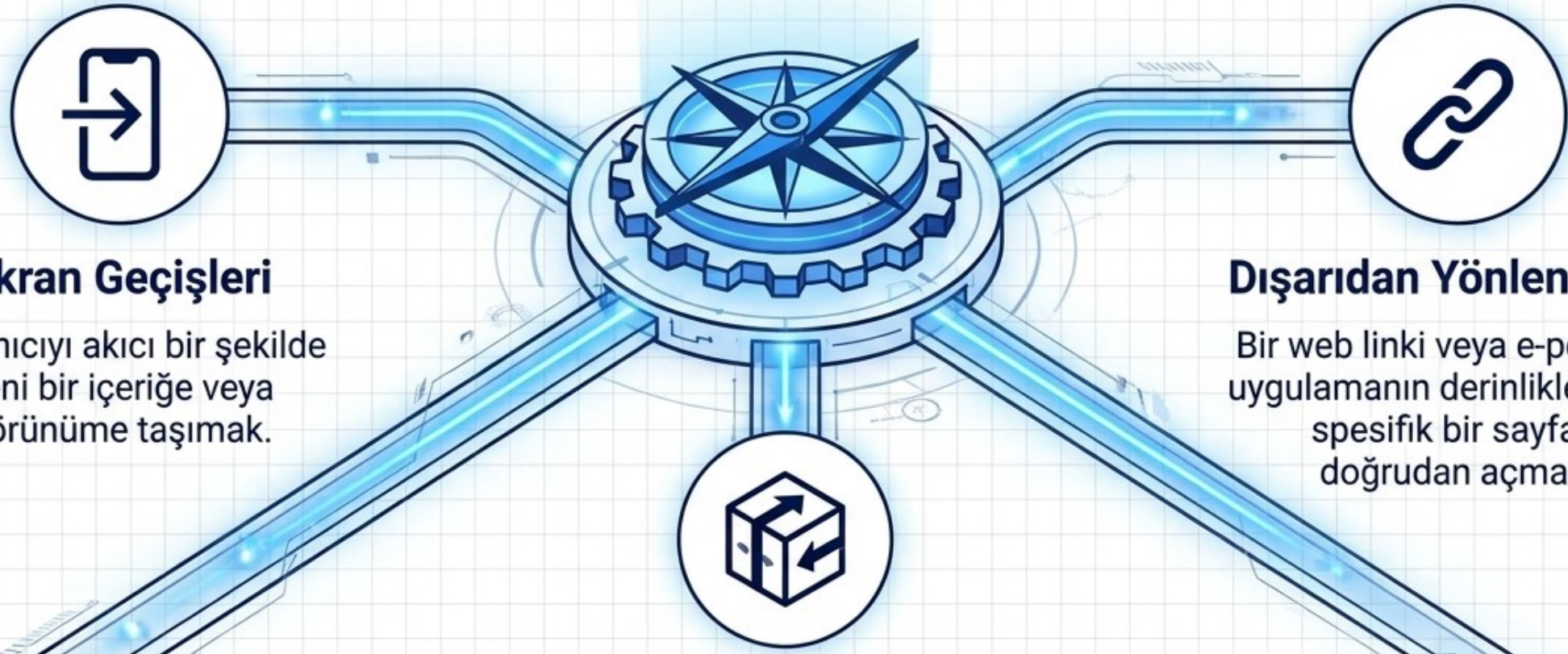
Platforma özgü ayarlar ve entegrasyon



Web URL Stratejisi

URL tabanlı navigasyon ve web uyumluluğu

Neden Navigasyona İhtiyaç Duyarız?



Ekran Geçişleri

Kullanıcıyı akıcı bir şekilde yeni bir içeriğe veya görünüme taşımak.

Dışarıdan Yönlendirme

Bir web linki veya e-posta ile uygulamanın derinliklerindeki spesifik bir sayfayı doğrudan açmak.

Veri Taşıma

Bir listedeki seçili elemanın detaylarını hedef ekrana aktarmak.

Temel Aktörler: Route ve Navigator



- **Navigator:**
  Route nesnelerini bir yığın mantığıyla yöneten ana bileşendir. Kullanıcının uygulamanın neresinden geldiğini hafızada tutar.
- **Route:**
  Geçiş yapılacak hedef ekranı temsil eder. Yığının içine eklenen ve çıkarılan temel yapı taşıdır.

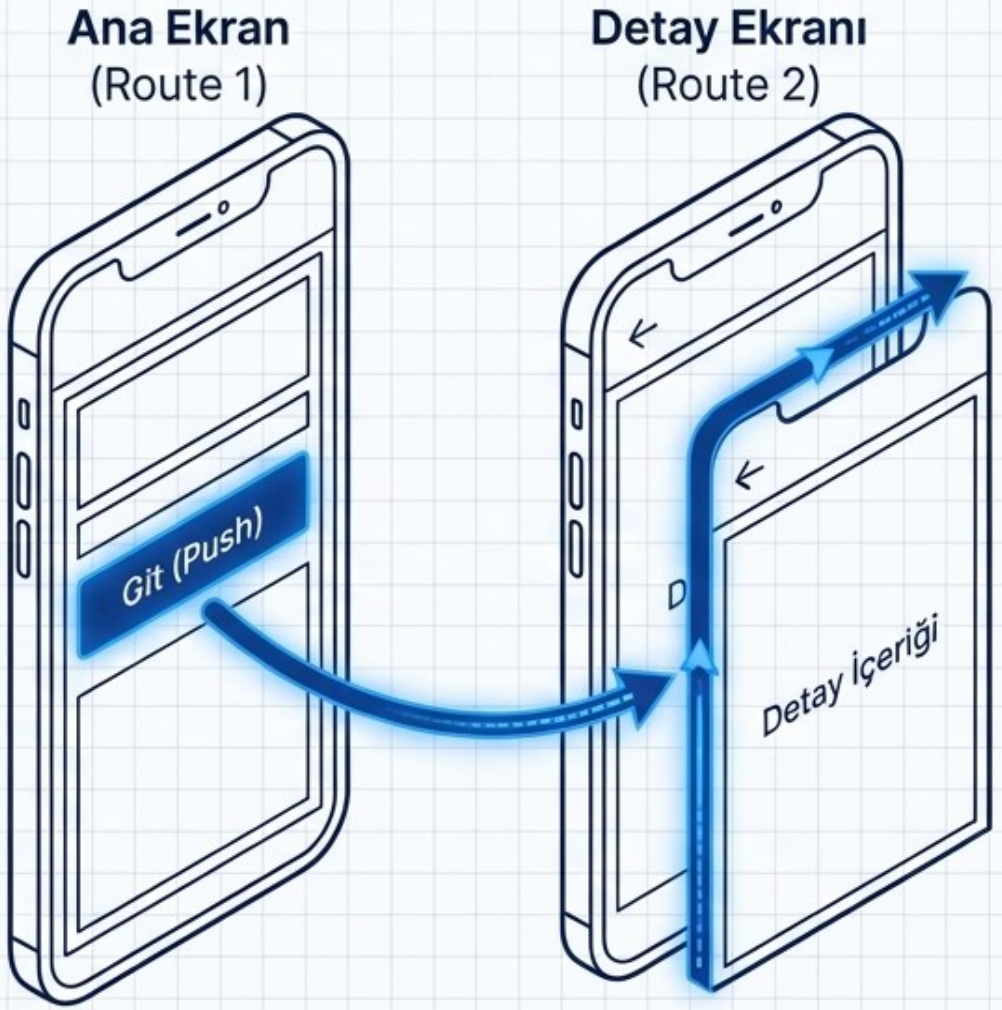
Doğru Navigasyon Aracını Seçmek

Senaryo	Navigator (Zorunlu)	Router (Bildirimsel)
Küçük Uygulamalar ve Basit Geçişler	✓ Uygun	✓ Uygun
Karmaşık Derin Bağlantı (Deep Linking)	✗ Önerilmez	✓ **Tam Destek**
Web URL & Geçmiş Senkronizasyonu	✗ Desteklenmez	✓ **Tam Destek**

Modern ve büyük ölçekli Flutter projelerinde bildirimsel (declarative) yönlendirme paketi olan go_router kullanımı standarttır.

Yeni Bir Ekrana Geçiş: Navigator.push()

```
Navigator.push(
  context,
  MaterialPageRoute(builder:
    (context) => const SecondRoute()),
  ),
);
```



Geri Dönüş: Navigator.pop()

```
onPressed: () {
  Navigator.pop(context);
}
```

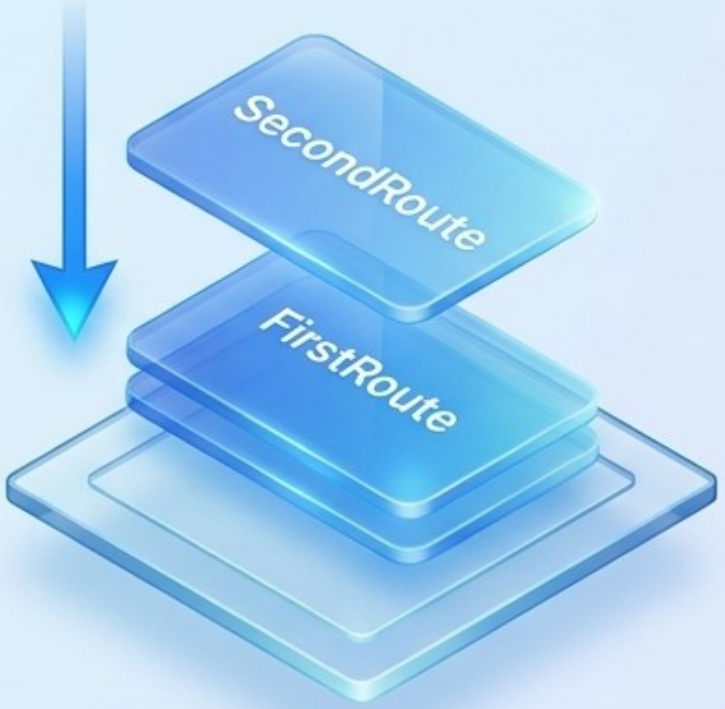


Yığın (Stack) Mimarisi Nasıl Çalışır?

Kural: LIFO
(Son Giren İlk Çıkar)



Boş yığına ilk rota eklenir.



Push: Yeni rota üstte birikir.



Pop: Üstteki rota uçarak çıkar, alttaki görünür.

Platforma Özgü Geçiş Karakteristikleri

MaterialPageRoute



Aşağıdan yukarıya hareket ve opaklaşma (fade).

CupertinoPageRoute



Sağdan sola doğru kayarak ekranı kaplama.

Flutter, hedef platformun doğal (native) gezinme hissini ilgili Route alt sınıflarıyla otomatik olarak sağlar.

Adlandırılmış Rotalar (Named Routes)

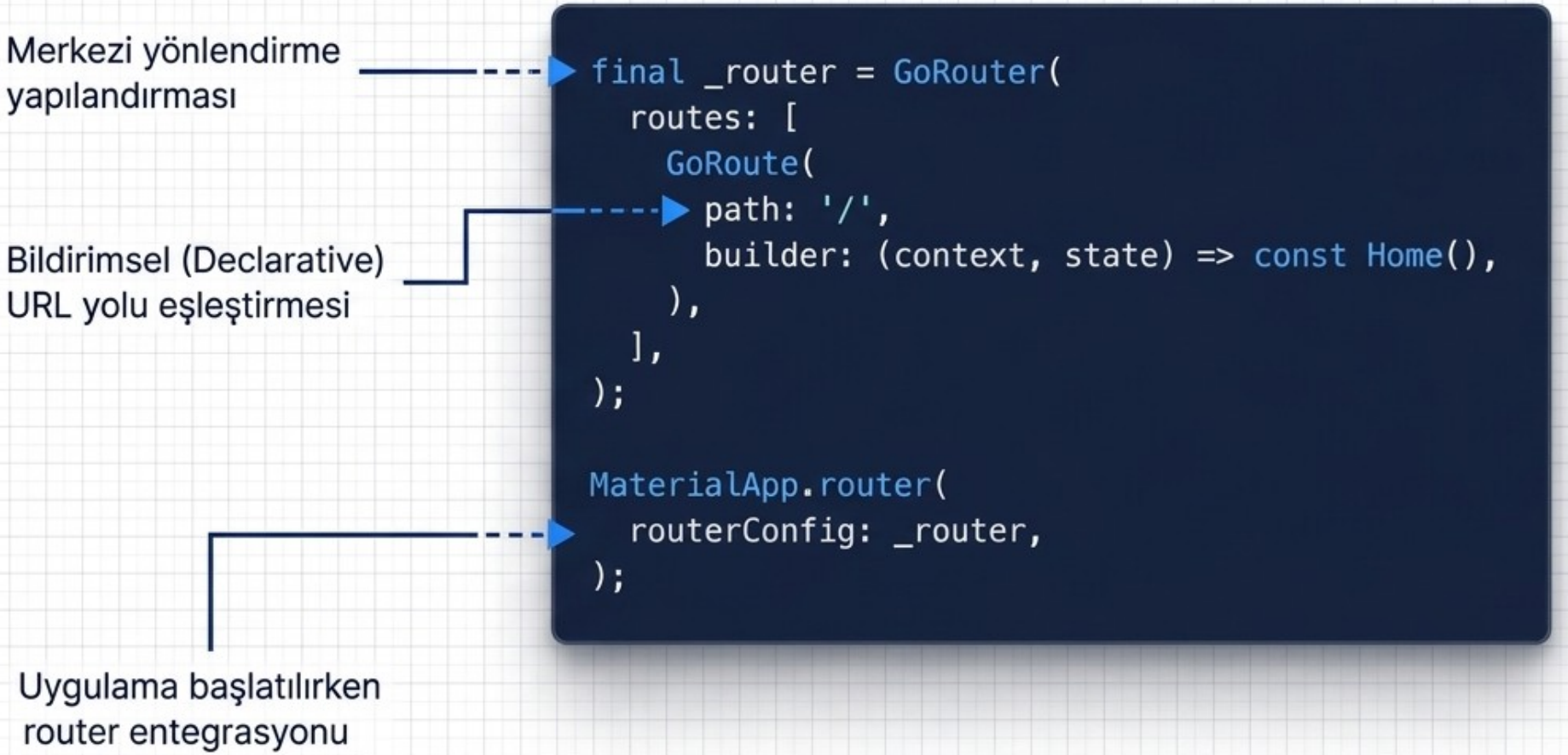
```
routes: {
  '/': (context) => First(),
  '/second': (context) => Second()
}
```

Neden Önerilmez? (Sınırlamalar)

- **1.** Derin bağlantılarda (Deep Links) özelleştirilemez.
- **2.** Web tarayıcısı ileri düğmesini desteklemez.
- **3.** Yeni bir derin bağlantı geldiğinde, kullanıcı nerede olursa olsun mevcut Navigator üzerine yeni bir Route atar.

Çoğu uygulamada pushNamed kullanmaktan kaçının!

Modern Yönlendirme Standardı: go_router



Yaklaşım Farkı: Zorunlu vs. Bildirimsel



Navigator (Zorunlu / Imperative)

Soru: NASIL yapılacağını söyler.

Mantık: “Şu ekranı al ve zorla yığının en üstüne ekle.” (push)

Adım adım manuel talimatlar gerektirir.



go_router (Bildirimsel / Declarative)

Soru: NE olacağını söyler.

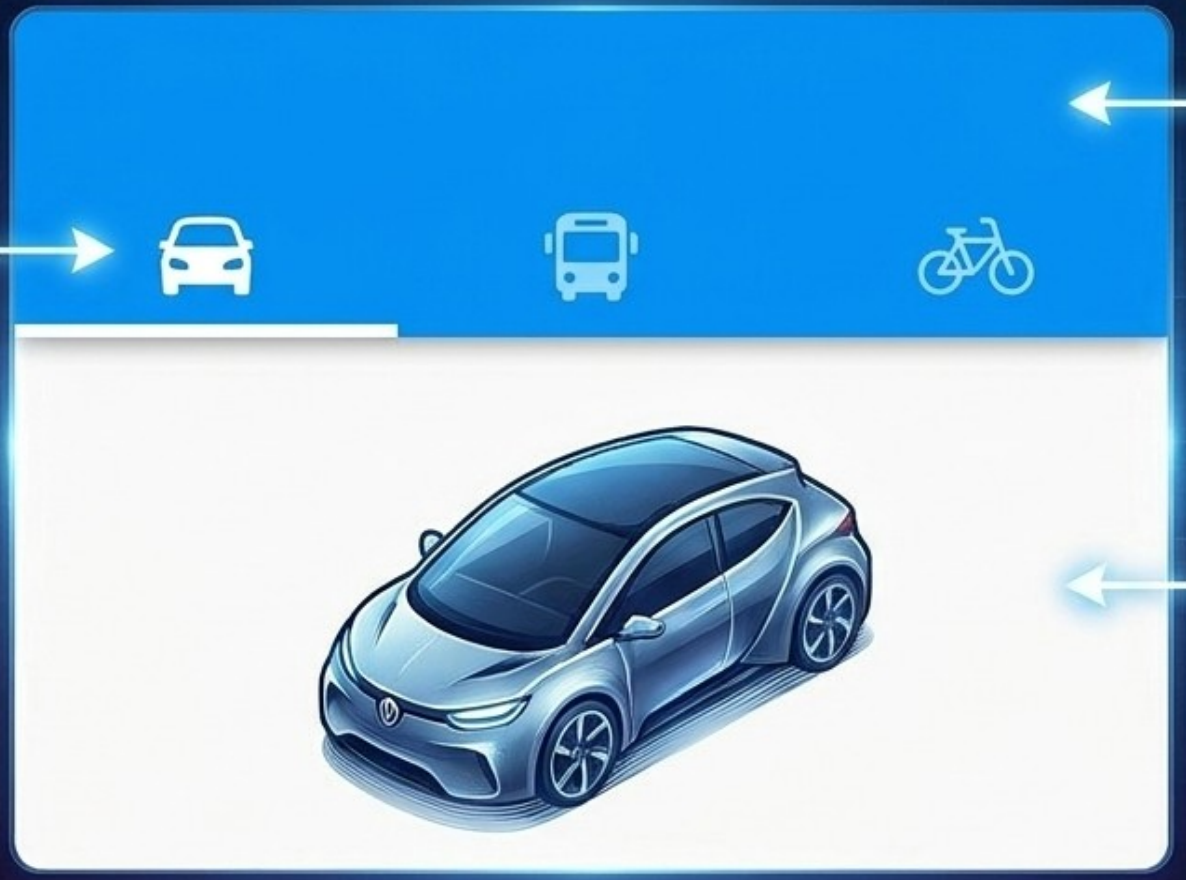
Mantık: “Uygulamanın durumu bu, bu duruma uygun ekran dizilimini göster.”

Sonuca odaklıdır, derin bağlantılar için her zaman tutarlıdır.

Sekmelerle Çalışma (Tabs)

AppBar

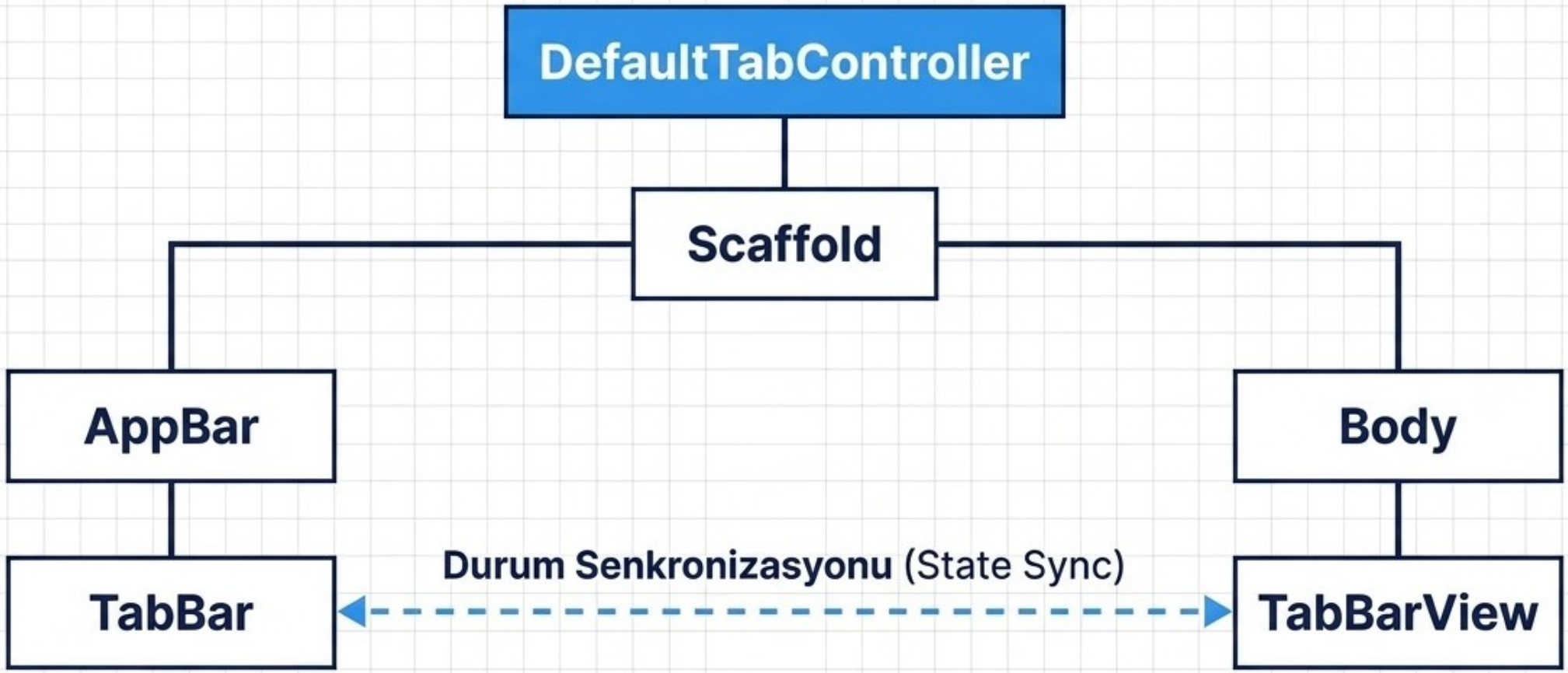
TabBar
(Tıklanabilir sekmeler)



TabBarView
(Değişen içerik ekranı)

Sekmelerin senkronize çalışması için bir TabController gereklidir.

DefaultTabController’ın Çalışma Mantığı



Uygulamada Sekmeler (Kod ve UI)

```
DefaultTabController(
  length: 3,
  child: Scaffold(
    appBar: AppBar(
    bottom: TabBar(
      tabs: [
        Tab(icon: Icon(Icons.directions_car)),
        Tab(icon: Icon(Icons.directions_transit)),
        Tab(icon: Icon(Icons.directions_bike)),
      ],
    ),
    body: TabBarView(...),
  ),
),
```



Rotalar Arasında Veri Aktarımı Senaryosu



Sadece ekranı değiştirmek yetmez, bağlamı (context) hedefe taşımak gerekir.

Senaryo: Todo (Yapılacaklar) Listesi ve Detay



Adım 2: Veriyi Hedefe Ulaştırma Yöntemleri

Yöntem A: Kurucu (Constructor) İle

```
class DetailScreen extends StatelessWidget {
  final Todo todo;

  const DetailScreen({super.key, required
this.todo});
}
```

Hedef ekran, veriyi kendi kurucu metodu üzerinden zorunlu parametre olarak talep eder.

Yöntem B: RouteSettings İle

```
final todo = ModalRoute.of(context)!.setting
 s.arguments as Todo;
```

Hedef ekran, Navigator.push sırasında iletilen argümanları ModalRoute üzerinden ayıklar.

Bir Ekrandan Veri Geri Döndürme



1. Bekleyiş: Ana Ekran 'await push' ile Seçim Ekranını açar ve sonucu bekler.



2. Karar: Seçim Ekranında Navigator.pop(context, 'Evet!') tetiklenir.



3. Sonuç: Ana Ekrana dönülür ve Snackbar ('Seçim: Evet!') gösterilir.

Yan Menü (Drawer) Nedir?

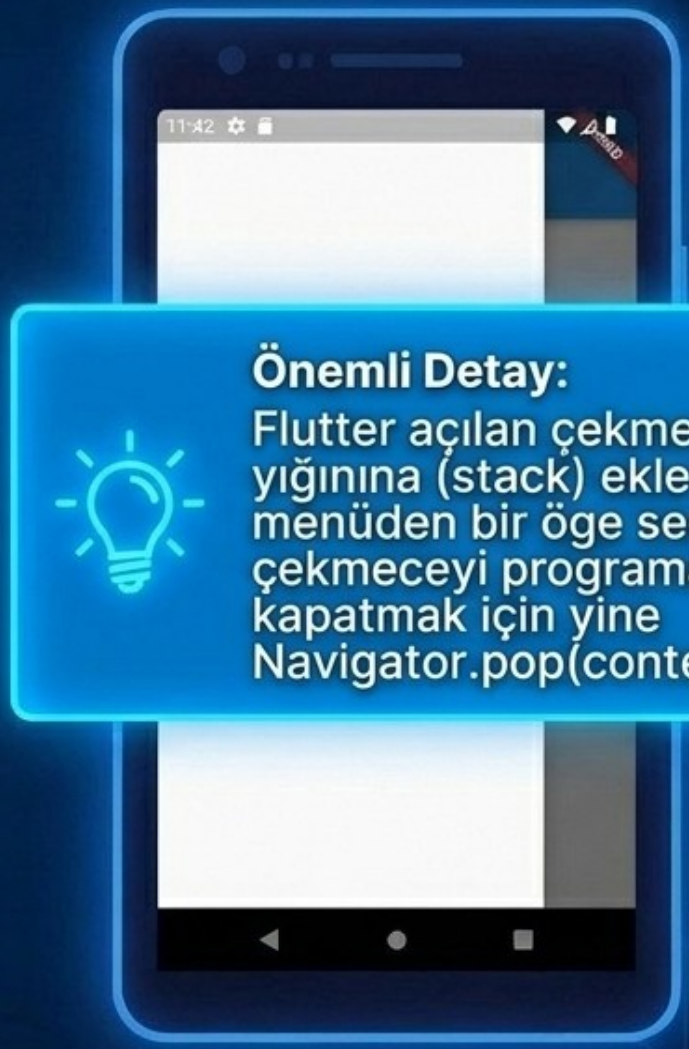
Material Design sisteminde, sekmeler (tabs) için yeterli alan kalmadığında kullanılan alternatif bir düzen (layout) yöntemidir.

Ekranın sol kenarından kayarak açılır ve genellikle uygulamanın ana gezinme köklerini barındırır.



Drawer Yönetimi (Kod ve Arayüz)

```dart
Scaffold(
  drawer: Drawer(
    child: ListView(
      children: [
        DrawerHeader(...),
        ListTile(...),
      ],
    ),
  ),
)
```



Önemli Detay:

Flutter açılan çekmeceyi de gezinme yığınına (stack) ekler. Kullanıcı menüden bir öge seçtiğinde çekmeceyi programatik olarak kapatmak için yine Navigator.pop(context) kullanılır.

Cupertino Sheet: iOS tarzı alt sayfa

Ekranın altından yükselen, aşağı çekilerek kapatılabilen modal görünüm.

```
showCupertinoSheet<void>(
  context: context,
  scrollableBuilder: (ctx, sc) =>
    SingleChildScrollView(
      controller: sc,
      child: const Text("Merhaba!"),
    ),
);
```

Modal içerik

Aşağı kaydırarak kapat



Kaynak: Flutter Docs – Navigation & Routing

Deep Linking: Bağlantıyla doğru ekrana

Bir URL, uygulamanın açılış ekranına değil belirli bir içeriğe götürür.

`https://ornek.com/details`



01

URL açılır

02

Rota eşleştirilir

03

Doğru ekran gösterilir

Kaynak: Flutter Docs – Navigation & Routing

URL'den ekrana: Router akışı

Router, gelen bağlantının yolunu ayrıştırıp uygun Navigator sayfalarını yapılandırır.

1 Bağlantı gelir

`/details`

→

2 Yol eşleştirilir

go_router

→

3 Sayfa oluşturulur

GoRoute

→

4 Hedef görünür

DetailsScreen

```
GoRoute(path: '/details',
  builder: (context, state) =>
    const DetailsScreen(),
)
```

Ekrana yönlendirme

```
context.go('/details');
```

Aynı URL, uygulama durumundan bağımsız olarak ilgili rotayı hedefler.

Kaynak: Flutter Docs – Navigation & Routing

Android App Links nasıl çalışır?

HTTP(S) bağlantısını doğrulanmış Android uygulamasında açma yaklaşımı.

01

AndroidManifest.xml

intent-filter ile kabul edilen bağlantı türü ve alan adı

02

assetlinks.json

Web alan adı ile Android uygulaması arasındaki ilişki

03

SHA-256 parmak izi

Uygulamanın imzalama sertifikasını doğrulama

Alan adı → **Doğrulama dosyası** → **Uygulama açılır**

Not: Bir App Link için doğrulanabilir bir web alan adı gerekir.

Kaynak: Flutter Docs – Navigation & Routing

iOS Universal Links: üç yapı taşı

iOS, uygulama–alan adı eşleşmesini Associated Domains ile doğrular.

01

Associated Domains

Xcode > Signing & Capabilities

`applinks:ornek.com`

02

apple-app-site-association

Web sunucusundaki ilişkilendirme dosyası

`/.well-known/...`

03

appID

Team ID + Bundle ID

`TEAMID.com.ornek.app`

Android `assetlinks.json`

iOS `apple-app-site-association`

Kaynak: Flutter Docs – Navigation & Routing

Web URL stratejileri: Hash mı, Path mi?

Aynı ekran için iki farklı URL gösterim yaklaşımı.

HASH (VARSAYILAN)

`ornek.dev/#/details`

Rota, URL'nin # sonrasındaki bölümünden okunur.

PATH (YOL)

`ornek.dev/details`

Daha temiz görünür; sunucuda ek yönlendirme ayarı gerekir.

`Path için: usePathUrlStrategy()`

Kaynak: Flutter Docs – Navigation & Routing

Tarayıcı geçmişi ve sunucu yapılandırması

Router, History API ile geri–ileri gezinmeyi adres çubuğuyla uyumlu tutar.



Path URL kullanıyorsak:

Sunucu route isteklerini index.html'e yönlendirmeli.

Alt klasörde yayımlıyorsak:

web/index.html içindeki base href güncellenmeli.

Kaynak: Flutter Docs – Navigation & Routing

Hangi navigasyon yöntemi ne zaman?

Senaryoya uygun aracı seçmek kodu ve kullanıcı deneyimini sadeleştirir.

Basit ekran geçişi	**Navigator.push() / pop()**	İki–üç ekran arasında ileri ve geri adımları.
Ekranlar arası veri	**Constructor / RouteSettings**	Seçilen nesnenin ayrıntılarını yeni ekrana ilet.
Üst sekmeler	**TabBar + TabBarView**	Aynı sayfa yapısında farklı içerikler göster.
Yan menü	**Scaffold + Drawer**	Ekran alanı dar olduğunda menü seçeneklerini topla.
URL ve Deep Linking	**Router + go_router**	Web adresi, doğrudan açılan sayfa ve gelişmiş akışlar.

Hatırla: Adlandırılmış rotalar her uygulamanın varsayılan tercihi olmak zorunda değil.

Kaynak: Flutter Docs – Navigation & Routing

FLUTTER • NAVIGATION & ROUTING

Gezinme artık daha anlamlı.

push / pop • veri akışı • sekmeler • deep link

Ekran	Arayüz	Bağlantı
Navigator	TabBar & Drawer	go_router

Sorular?

Nilay Özçelik • **Yüsra Paçal** • **Yağmur Öztekin**

Kaynak: Flutter Docs — Navigation & Routing (Türkçe çeviri)